

Machine Learning Techniques

Unit I: Data Pre-processing

M.Sc. Quantitative Finance

Instructor: Pranav T V

Department of Statistics, Pondicherry University

Outline

- 1 Introduction to Machine Learning
- 2 Types of Variables
- 3 Data Pre-processing
- 4 Train-Test-Validation Split
- 5 Missing Value Imputation
- 6 Imbalanced Categorical Data
- 7 Summary and Revision

Learning Objectives

By the end of this unit, students will be able to:

- Understand the fundamentals and importance of data pre-processing
- Identify different types of variables encountered in financial datasets
- Prepare training, testing, and validation datasets correctly
- Perform missing value imputation using appropriate techniques
- Understand and address class imbalance in categorical data
- Apply the concepts learnt using Python (pandas, numpy, scikit-learn)

What is Machine Learning?

Definition: Machine Learning (ML) is a branch of Artificial Intelligence in which a computer system **learns patterns from data** rather than being explicitly programmed with rules.



Finance Intuition: Instead of hand-coding rules like “if credit score < 600, reject loan”, an ML model *learns* such patterns automatically from historical loan data.

Why Machine Learning?

Traditional Programming

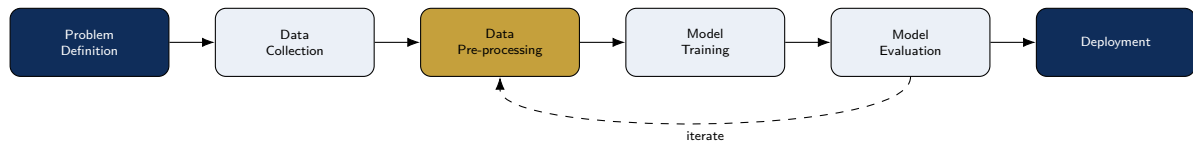
- Rules written by humans
- Struggles with complex, high-dimensional patterns
- Hard to update as markets evolve

Machine Learning

- Rules learned from data
- Captures non-linear, complex relationships
- Adapts as new data arrives

Example: Predicting stock price movement depends on hundreds of interacting factors (volume, volatility, macro news, sentiment) – far too complex to hand-code as rules.

Machine Learning Workflow



This unit focuses entirely on the **Data Pre-processing** stage – arguably the most time-consuming and important step (often 60–80% of a real project).

Machine Learning in Finance

Area	ML Application
Credit Risk	Predicting probability of loan default
Fraud Detection	Flagging suspicious card transactions
Algorithmic Trading	Predicting short-term price movements
Portfolio Management	Robo-advisory, risk-based allocation
Insurance	Predicting claim likelihood, pricing premiums
Customer Analytics	Segmenting customers for targeted products

Advantages and Challenges

Advantages

- Handles large, complex datasets
- Uncovers hidden non-linear patterns
- Improves with more data
- Automates repetitive analytical tasks

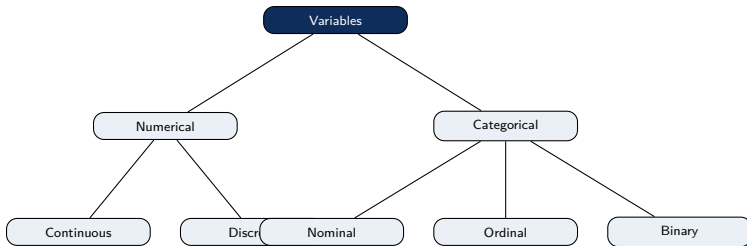
Challenges

- Requires large, clean data
- Risk of overfitting
- Interpretability (“black box”)
- Regulatory and ethical concerns in finance

Practice Questions: Introduction

- ❶ [True/False] Machine learning always requires explicit rule-based programming.
- ❷ [Fill in the blank] ML models improve their performance as more _____ is provided.
- ❸ [Short Answer] Define Machine Learning in your own words.
- ❹ [Conceptual] Why is ML particularly useful in quantitative finance?
- ❺ [MCQ] Which of the following is *not* a stage in the ML workflow? (a) Data Collection (b) Model Training (c) Manual Rule Writing (d) Model Evaluation
- ❻ [Discussion] Discuss one risk of using ML models for credit scoring.
- ❼ [Conceptual] What percentage of a real-world ML project is typically spent on data pre-processing?
- ❽ [Short Answer] Give two examples of ML applications in banking.
- ❾ [MCQ] A “black box” model refers to a model that is: (a) easy to interpret (b) hard to interpret (c) always accurate (d) never used in finance
- ❿ [Python Practice] List three Python libraries commonly used for ML.

Types of Variables: Overview



Numerical Variables

Numerical Variables: Variables that represent quantities and support arithmetic operations.

- **Continuous:** Can take any value within a range (e.g., *stock price, portfolio return, volatility*)
- **Discrete:** Countable, whole-number values (e.g., *number of trades per day, number of defaults*)

Finance Example: Daily closing price of a stock (continuous) vs. number of transactions in a portfolio (discrete).

Categorical Variables

Categorical Variables: Variables that represent categories or groups, not naturally numeric.

- **Nominal:** No inherent order (e.g., *sector: banking, IT, pharma*)
- **Ordinal:** Natural order exists (e.g., *credit rating: AAA > AA > A > BBB*)
- **Binary:** Only two categories (e.g., *loan default: yes/no*)

Comparison: Numerical vs. Categorical

Feature	Numerical	Categorical
Nature	Quantitative	Qualitative
Arithmetic Ops	Meaningful	Not meaningful
Example (Finance)	Stock return (%)	Credit rating
Common Encoding	Scaling/Normalization	One-hot / Label encoding

Python Example: Identifying Variable Types

```
import pandas as pd

data = {
    'stock': ['TCS', 'INFY', 'HDFC', 'RELIANCE'],
    'sector': ['IT', 'IT', 'Banking', 'Energy'],
    'credit_rating': ['AAA', 'AA', 'AAA', 'A'],
    'close_price': [3450.5, 1520.2, 1600.8, 2450.0],
    'num_trades': [1200, 950, 1100, 2000]
}

df = pd.DataFrame(data)
print(df.dtypes)
```

```
stock object
sector object
credit_rating object
close_price float64
num_trades int64
dtype: object
```

Interpretation: close_price is continuous numerical, num_trades is discrete numerical, sector and credit_rating are categorical.

Practice Questions: Types of Variables

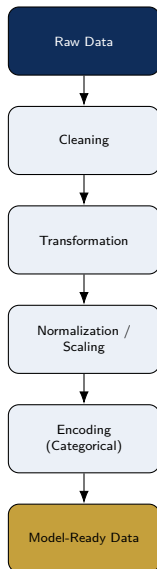
- 1 **[True/False]** Credit rating is an example of a nominal variable.
- 2 **[Fill in the blank]** A variable that can take infinitely many values in a range is called _____.
- 3 **[Short Answer]** Give two examples each of nominal and ordinal variables in finance.
- 4 **[Conceptual]** Why does the distinction between variable types matter for preprocessing?
- 5 **[MCQ]** Loan default status (yes/no) is: (a) Continuous (b) Discrete (c) Binary categorical (d) Ordinal
- 6 **[Numerical]** Classify: portfolio beta, number of shares held, investment grade.
- 7 **[Discussion]** Can a discrete variable ever be treated as categorical? Explain.
- 8 **[Python Practice]** Write code to list all categorical columns of a DataFrame.
- 9 **[MCQ]** Which encoding is typically used for nominal variables? (a) Label encoding (b) One-hot encoding (c) Z-score (d) Min-max scaling
- 10 **[Short Answer]** Why is sector classification usually nominal and not ordinal?

Why Data Pre-processing?

Motivation: Real-world financial data is rarely clean – it contains missing values, inconsistent scales, categorical text, and outliers. Feeding raw data into an ML model leads to poor and unreliable predictions.

- “Garbage in, garbage out”
- Different features (e.g., price in thousands vs. return in decimals) need comparable scales
- Algorithms like KNN, SVM, Neural Networks are sensitive to scale
- Missing or inconsistent entries can bias the model

Data Pre-processing Pipeline



Key Pre-processing Steps

Step	Description
Cleaning	Removing duplicates, correcting errors, handling missing values
Transformation	Converting data into a suitable format (e.g., log returns from prices)
Normalization/Scaling	Bringing numerical features to a common scale (e.g., Min-Max, Z-score)
Encoding	Converting categorical variables into numeric form (e.g., one-hot)

Normalization: The Mathematics

Min-Max Scaling: Rescales a variable x to the range $[0, 1]$:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- x = original value
- x_{min}, x_{max} = minimum and maximum values of that feature
- x' = scaled value between 0 and 1

Z-score Standardization: Rescales using mean and standard deviation:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation of the feature.

Python Example: Scaling and Encoding

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

df = pd.DataFrame({
    'close_price': [3450.5, 1520.2, 1600.8, 2450.0],
    'sector': ['IT', 'IT', 'Banking', 'Energy']
})

scaler = MinMaxScaler()
df['price_scaled'] = scaler.fit_transform(df[['close_price']])

df_encoded = pd.get_dummies(df, columns=['sector'])
print(df_encoded)
```

```
   close_price  price_scaled  sector_Banking  sector_Energy  sector_IT
0  3450.5      1.000000      False      False      True
1  1520.2      0.000000      False      False      True
2  1600.8      0.041775       True      False      False
3  2450.0      0.481453      False       True      False
```

Practice Questions: Data Pre-processing

- 1 [True/False] Scaling is unnecessary for tree-based models such as Decision Trees.
- 2 [Fill in the blank] Converting categorical text into numeric columns is called _____.
- 3 [Short Answer] Why do KNN and SVM require feature scaling?
- 4 [Numerical] If $x = 80$, $x_{min} = 20$, $x_{max} = 100$, compute the Min-Max scaled value.
- 5 [Conceptual] Explain the difference between normalization and standardization.
- 6 [MCQ] One-hot encoding creates: (a) One column (b) A column per category (c) A single scaled column (d) No new columns
- 7 [Discussion] What issues arise from one-hot encoding a variable with 500 categories?
- 8 [Python Practice] Write code to standardize a column using StandardScaler.
- 9 [MCQ] Log transformation of stock prices is typically used to: (a) Increase skewness (b) Reduce skewness (c) Remove outliers (d) Encode categories
- 10 [Short Answer] List the four key steps in the pre-processing pipeline.

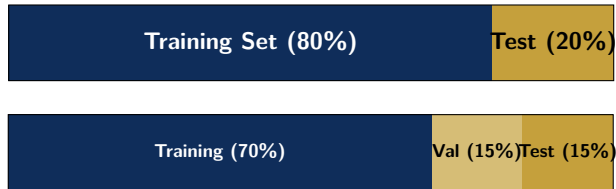
Why Split the Data?

Motivation: We must evaluate a model on data it has **never seen** to know if it generalizes, rather than merely memorizing the training data.

- **Training set:** used to fit (train) the model
- **Testing set:** used to evaluate final model performance
- **Validation set:** used during development to tune hyperparameters

Finance Example: Train a default-prediction model on 2015–2021 loan data, and test it on unseen 2022 loans to check real-world reliability.

Train-Test Split Visualization



Common splits: **80–20**, **70–30**, or **70–15–15** (train–validation–test).

Cross-Validation (Brief Introduction)

k-Fold Cross-Validation: The training data is split into k equal parts (“folds”). The model is trained on $k - 1$ folds and validated on the remaining fold – repeated k times, rotating the validation fold.

- Gives a more robust estimate of model performance
- Reduces dependence on a single lucky/unlucky split
- Common choice: $k = 5$ or $k = 10$

Python Example: Train-Test Split

```
from sklearn.model_selection import train_test_split
import pandas as pd

X = df_encoded.drop(columns=['close_price'])
y = df_encoded['close_price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Train size:", X_train.shape[0])
print("Test size :", X_test.shape[0])
```

```
Train size: 3
Test size : 1
```

Practice Questions: Train-Test-Validation

- 1 [True/False] The test set should be used to tune model hyperparameters.
- 2 [Fill in the blank] The set used to fit model parameters is called the _____ set.
- 3 [Short Answer] Why is an 80-20 split more common than 50-50 in many finance applications?
- 4 [Conceptual] What problem occurs if we evaluate a model only on its training data?
- 5 [MCQ] In 10-fold cross-validation, the model is trained: (a) once (b) 5 times (c) 10 times (d) 100 times
- 6 [Numerical] If a dataset has 10,000 rows and we use a 70-30 split, how many rows go to testing?
- 7 [Discussion] Why might a purely random train-test split be problematic for time-series stock data?
- 8 [Python Practice] Modify the code above to use a 70-30 split.
- 9 [MCQ] Validation sets are primarily used for: (a) Final reporting (b) Hyperparameter tuning (c) Data cleaning (d) Encoding
- 10 [Short Answer] Define cross-validation in your own words.

Types of Missing Data

Type	Meaning	Finance Example
MCAR	Missing Completely at Random – missingness unrelated to any variable	Data entry system randomly fails to log some trades
MAR	Missing at Random – missingness related to <i>other observed</i> variables	Younger customers more likely to skip the “income” field
MNAR	Missing Not at Random – missingness related to the <i>missing value itself</i>	High-risk borrowers deliberately omit debt information

Methods of Missing Value Imputation

- **Mean/Median/Mode Imputation:** Replace missing value with the column's average/median/most frequent value
- **KNN Imputation:** Use values from the k most similar rows
- **Forward Fill:** Carry the last observed value forward (common in time series)
- **Backward Fill:** Carry the next observed value backward
- **Interpolation:** Estimate missing values using surrounding data points (e.g., linear interpolation)

Mean Imputation: The Mathematics

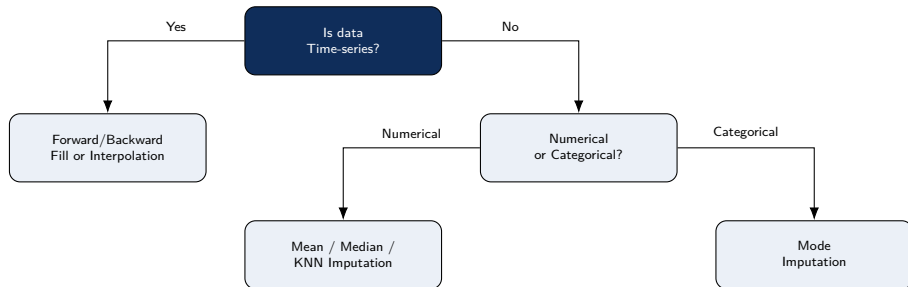
Mean Imputation Formula: Missing values are replaced with the arithmetic mean of the observed values:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- x_i = the i -th observed (non-missing) value
- n = the number of observed values
- $\bar{x}$ = the mean (average), which replaces every missing entry

Intuition: If a stock's daily volume is missing for one day, we can fill it with the average volume across all other observed days.

Decision Flowchart: Choosing an Imputation Method



Comparison of Imputation Methods

Method	Advantages	Disadvantages
Mean/Median	Simple, fast	Distorts variance, ignores relationships
Mode	Works for categorical data	Can over-represent common category
KNN	Uses relationships between features	Computationally expensive on large data
Forward/Backward Fill	Good for time series	Assumes recent value is representative
Interpolation	Smooth estimates for trends	Not suitable for non-sequential data

Python Example: Missing Value Imputation

```
import pandas as pd
import numpy as np
from sklearn.impute import KNNImputer

df = pd.DataFrame({
    'price': [100, np.nan, 102, 105, np.nan],
    'volume': [1000, 1100, np.nan, 1050, 980]
})

# Mean imputation
df_mean = df.fillna(df.mean())

# KNN imputation
imputer = KNNImputer(n_neighbors=2)
df_knn = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

print(df_mean)
print(df_knn)
```

```
   price volume
0  100.00  1000.00
1  102.33  1100.00
2  102.00  1032.50
3  105.00  1050.00
```


Practice Questions: Missing Value Imputation

- 1 [True/False] MNAR missingness can be safely ignored without bias.
- 2 [Fill in the blank] Replacing a missing value with the most frequent category is called _____ imputation.
- 3 [Short Answer] Distinguish between MCAR and MAR with an example.
- 4 [Numerical] Given observed prices {100, 102, 105, 98}, compute the mean used for imputation.
- 5 [Conceptual] Why might mean imputation understate the true variance of a variable?
- 6 [MCQ] KNN imputation estimates a missing value using: (a) Global mean (b) Nearest neighbors (c) Mode only (d) Random values
- 7 [Discussion] Why is forward-fill preferred over mean imputation for stock price time series?
- 8 [Python Practice] Write code to impute missing values in a column using the median.
- 9 [MCQ] Missingness that depends on the missing value itself is: (a) MCAR (b) MAR (c) MNAR (d) None
- 10 [Short Answer] List two disadvantages of KNN imputation.

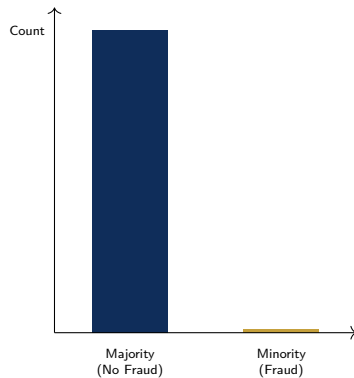
Class Imbalance: The Problem

Definition: Class imbalance occurs when one category of the target variable is far more frequent than another.

Finance Example: In fraud detection, genuine transactions may be 99.9% of the data while fraudulent transactions are only 0.1%.

- Models tend to become biased toward the majority class
- A model predicting “no fraud” every time would achieve 99.9% accuracy but be useless
- Requires special sampling and evaluation strategies

Imbalance Visualization



Sampling Methods

- **Over-sampling:** Increase the number of minority class samples (e.g., duplicate or synthesize)
- **Under-sampling:** Reduce the number of majority class samples
- **SMOTE (Synthetic Minority Over-sampling Technique):** Creates new synthetic minority examples by interpolating between existing minority points, rather than simple duplication

Intuition: SMOTE doesn't just copy fraud cases – it generates plausible “new” fraud-like data points between existing ones.

Confusion Matrix (Brief Introduction)

		Predicted	
		Positive	Negative
Actual	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

Used to evaluate classification models – especially important with imbalanced data, since plain **accuracy** can be misleading.

Evaluation After Balancing

- **Accuracy** alone is misleading for imbalanced problems
- **Precision:** Of predicted positives, how many were correct?
- **Recall:** Of actual positives, how many did we find?
- **F1-score:** Harmonic mean of Precision and Recall

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

where TP , FP , FN are counts from the confusion matrix.

Python Example: Handling Imbalanced Data

```
from imblearn.over_sampling import RandomOverSampler
from collections import Counter
import pandas as pd

X = pd.DataFrame({'amount': [10, 20, 15, 1000, 25, 30]})
y = [0, 0, 0, 1, 0, 0] # 1 = Fraud (minority)

print("Before:", Counter(y))

ros = RandomOverSampler(random_state=42)
X_res, y_res = ros.fit_resample(X, y)

print("After :", Counter(y_res))
```

```
Before: Counter({0: 5, 1: 1})
After : Counter({0: 5, 1: 5})
```

Practice Questions: Imbalanced Data

- 1 [True/False] High accuracy always indicates a good model on imbalanced data.
- 2 [Fill in the blank] SMOTE generates _____ minority samples rather than duplicating existing ones.
- 3 [Short Answer] Why is fraud detection a classic example of class imbalance?
- 4 [Numerical] If $TP=40$, $FP=10$, $FN=5$, compute Precision and Recall.
- 5 [Conceptual] Explain the difference between over-sampling and under-sampling.
- 6 [MCQ] Under-sampling reduces: (a) Minority class (b) Majority class (c) Both classes equally (d) Neither
- 7 [Discussion] What is a potential drawback of under-sampling the majority class?
- 8 [Python Practice] Write code to apply `RandomUnderSampler` instead of over-sampling.
- 9 [MCQ] F1-score is the: (a) Sum of precision and recall (b) Harmonic mean of precision and recall (c) Product of TP and TN (d) Same as accuracy
- 10 [Short Answer] Why can accuracy be misleading for a 99%-1% imbalanced dataset?

Summary

- Data pre-processing transforms raw, messy financial data into a model-ready format
- Variables can be numerical (continuous/discrete) or categorical (nominal/ordinal/binary)
- Data must be split into training, validation, and testing sets to evaluate generalization
- Missing values can be MCAR, MAR, or MNAR, and are handled via mean/median/mode, KNN, or time-series-specific methods
- Imbalanced categorical data (e.g., fraud, default) requires over-/under-sampling or SMOTE, with careful evaluation using precision, recall, and F1-score

Key Takeaways

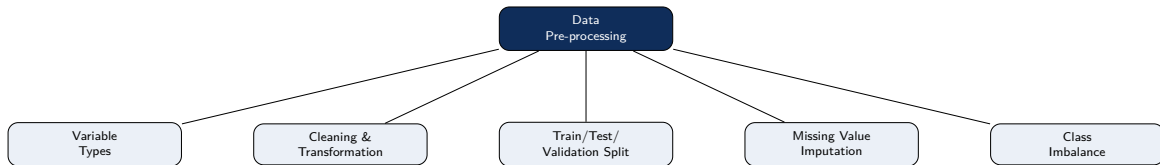
Pre-processing is 60–80% of ML work

Always split data before modeling

Handle missing data based on its mechanism (MCAR/MAR/MNAR)

Address class imbalance carefully – accuracy alone is not enough

Mind Map: Unit I



Important Formula Sheet

$$\text{Mean: } \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\text{Min-Max Scaling: } x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

$$\text{Z-score: } z = \frac{x - \mu}{\sigma}$$

$$\text{Precision: } \frac{TP}{TP + FP} \quad \text{Recall: } \frac{TP}{TP + FN}$$

Common Mistakes

- Scaling the entire dataset *before* splitting (causes data leakage)
- Using mean imputation blindly on MNAR data
- Ignoring class imbalance and relying only on accuracy
- Applying one-hot encoding to high-cardinality variables without care
- Forgetting to shuffle data before splitting non-time-series datasets

Interview Questions

- ❶ What is data leakage, and how can improper scaling cause it?
- ❷ How would you handle a dataset with 95% missing values in one column?
- ❸ Explain SMOTE to a non-technical interviewer.
- ❹ Why is stratified sampling important for imbalanced classification?
- ❺ What is the difference between validation and test sets in an interview scenario?

Revision Questions

- ➊ Summarize the complete data pre-processing pipeline in your own words.
- ➋ Compare MCAR, MAR, and MNAR with one example each.
- ➌ Explain why validation sets are needed in addition to test sets.
- ➍ Describe how SMOTE differs from simple over-sampling.
- ➎ List three real finance datasets where imbalance is common.

Assignment

Conceptual Questions (10): Cover variable types, missing data mechanisms, scaling, splitting, and imbalance concepts.

Numerical Questions (10): Compute means, Min-Max/Z-score values, precision/recall from given confusion matrices.

Python Exercises (5):

- 1 Load a CSV of stock prices and identify variable types
- 2 Impute missing values using mean and KNN, and compare results
- 3 Perform an 80-20 train-test split on a credit dataset
- 4 One-hot encode a categorical “sector” column
- 5 Apply SMOTE to a fraud dataset and compare class counts before/after

Finance Case Studies (2): (i) Pre-processing a bank loan default dataset; (ii) Pre-processing daily stock price data for a trading model.

Mini Project: Stock Price Data Pre-processing

Objective: Prepare a clean, model-ready dataset from raw daily stock price data.

Dataset: Daily OHLC (Open, High, Low, Close) prices and volume for a listed stock, with some missing entries.

Steps:

- 1 Load data and inspect variable types
- 2 Identify and impute missing values (mean/interpolation)
- 3 Engineer a “daily return” feature from close prices
- 4 Scale numerical features using Min-Max or Z-score
- 5 Split into training and testing sets (80-20)

Suggested Extensions: Add technical indicators (moving average, RSI); handle multiple stocks; explore SMOTE if building an up/down classification target.

Further Reading

- Hao Ni, Xin Dong, Jinson Zheng, Guangxi Yu – *An Introduction to Machine Learning in Quantitative Finance*, World Scientific, 2021
- Mathew F. Dixon, Igor Halperin, Paul Bilokon – *Machine Learning in Finance: From Theory to Practice*, Springer, 2020
- Saurav Singla – *Machine Learning for Finance*, BPB Publications

Thank You

End of Unit I: Data Pre-processing

Pranav T V | Department of Statistics | Pondicherry University